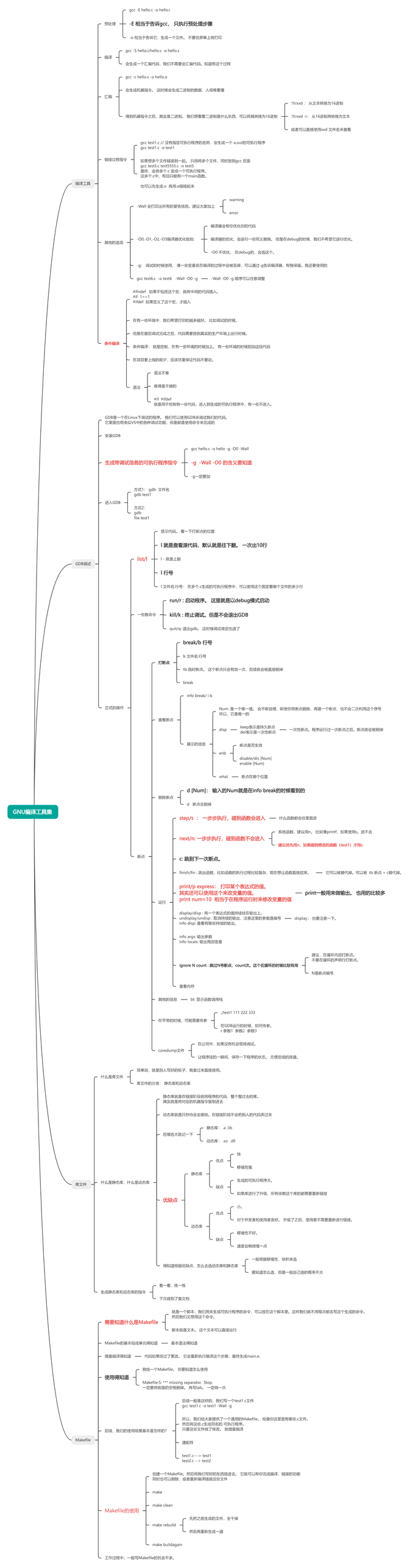
GNU编译工具集
编译工具
预处理
gcc -E hello.c -o hello.i
-E 相当于告诉gcc， 只执行预处理步骤
-o 相当于告诉它，生成一个文件。 不要往屏幕上倒打印
编译
gcc -S hello.i/hello.c -o hello.s
会生成一个汇编代码，我们不需要会汇编代码。知道有这个过程
汇编
gcc -c hello.s -o hello.o
会生成机器指令， 这时候会生成二进制的数据，人很难看懂
得到机器指令之后，其实是二进制。 我们想看看二进制是什么东西。可以将其转换为16进制
:%!xxd : 从文本转换为16进制
:%!xxd -r: 从16进制再转换为文本
或者可以直接使用xxd 文件名来查看
链接过程指令
gcc test1.c // 没有指定可执行程序的名称，会生成一个 a.out的可执行程序
gcc test1.c -o test1
如果想多个文件链接到一起。 只用将多个文件，同时放到gcc 后面
gcc test5.c test5555.c -o test5
最终，会将多个.c 变成一个可执行程序。
这多个.c中，有且只能有一个main函数。
也可以先生成.o 再用.o链接起来
其他的选项
-Wall 会打印出所有的警告信息。建议大家加上
warning
error
-O0, -O1, -O2, -O3编译器优化级别：
编译器会帮你优化你的代码
编译器的优化。会进行一些同义替换。但是在debug的时候，我们不希望它进行优化。
-O0 不优化， 在debug的，会选这个。
-g: 调试的时候使用， 像一些变量名在编译的过程中会被丢掉，可以通过-g告诉编译器，帮我保留。我还要使用的
gcc test5.c -o test6 -Wall -O0 -g —— -Wall -O0 -g 顺序可以任意调整
条件编译
#ifndef 如果不包括这个宏，就将中间的代码插入。
#if 1 --> 1
#ifdef 如果定义了这个宏，才插入
在有一些环境中，我们希望打印的越多越好。 比如调试的时候。
但是在最后调试完成之后，代码需要放到真实的生产环境上运行时候。
条件编译： 就是控制，在有一些环境的时候加上。 有一些环境的时候别加这段代码
在项目要上线的前夕，应该尽量保证代码不要动。
语法
语法不难
难得是干啥的
#if #ifdef
就是用于控制有一些代码，进入到生成的可执行程序中，有一些不进入。
GDB调试
GDB是一个在Linux下调试的程序。 我们可以使用GDB来调试我们的代码。
它里面也有类似VS中的各种调试功能。 但是都是使用命令来完成的
安装GDB
生成带调试信息的可执行程序指令
gcc hello.c -o hello -g -O0 -Wall
-g -Wall -O0 的含义要知道
-g一定要加
进入GDB
方式1： gdb 文件名
gdb test1
方式2：
gdb
file test1
正式的操作
list/l
显示代码。看一下打断点的位置
l 就是查看源代码，默认就是往下翻。 一次出10行
l - 就是上翻
l 行号
l 文件名:行号： 在多个.c生成的可执行程序中，可以使用这个固定看哪个文件的多少行
一些敢命令
run/r : 启动程序。 这里就是以debug模式启动
kill/k : 终止调试。但是不会退出GDB
quit/q: 退出gdb。 这时候调试肯定也退了
断点
打断点
break/b 行号
b 文件名:行号
tb 临时断点。 这个断点只会有效一次，后续就会被直接删掉
break
查看断点
info break/ i b
展示的信息
Num: 是一个唯一值。 会不断自增，即使你将断点删除，再建一个断点，也不会二次利用这个序号
所以，它是唯一的
disp —— keep表示是持久断点
del表示是一次性断点 —— 一次性断点。程序运行过一次断点之后，断点就会被删掉
enb —— 断点是否生效
disable/dis [Num]
enable [Num]
what —— 断点在哪个位置
删除断点
d [Num]： 输入的Num就是在info break的时候看到的
d 断点全删掉
运行
step/s ： 一步步执行，碰到函数会进入 —— 什么函数都会往里面进
next/n: 一步步执行，碰到函数不会进入 —— 系统函数，建议用n。 比如像printf，如果使用s，进不去
建议优先用n，如果碰到想进的函数（test1）才用s
c 跳到下一次断点。
finish/yfin：跳出函数，比如函数的执行过程比较复杂，现在想让函数直接结束。 —— 它可以被替代掉。可以被 tb 断点 + c替代掉。
print/p express： 打印某个表达式的值。
其实还可以使用这个来改变量的值。 —— print一般用来做输出， 也用的比较多
print num=10 相当于在程序运行时来修改变量的值
display/disp：将一个表达式的值持续挂在输出上。
undisplay/undisp：取消持续的输出。 注意这里的参数是编号 —— display：也要注意一下。
info disp: 查看有哪些持续的输出。
info args: 输出参数
info locals: 输出局部变量
ignore N count : 跳过N号断点，count次。这个在循环的时候比较有用
建议，在循环内部打断点，
不要在循环的声明时打断点。
N是断点编号
查看内存
其他的信息 —— bt: 显示函数调用栈
在平常的时候，可能需要传参
./test1 111 222 333
在GDB运行的时候，如何传参。
r 参数1 参数2 参数3
coredump文件
在公司中，如果没有机会现场调试。
让程序挂的一瞬间，保存一下程序的状态。 方便后续的排查。
库文件
什么是库文件
简单说，就是别人写好的轮子，我拿过来直接使用。
库文件的分类： 静态库和动态库
什么是静态库，什么是动态库
静态库就是在链接阶段就将程序的代码，整个整过去的库。
其实就是将对应的机器指令复制进去
动态库就是只存待会去哪找。在链接阶段不会把别人的代码弄过来
后缀名大致记一下
静态库： .a .lib
动态库： .so .dll
优缺点
静态库
优点
块
移植性强
缺点
生成的可执行程序大。
如果库进行了升级，所有依赖这个库的都需要重新链接
动态库
优点
小。
对于开发者和使用者友好。 升级了之后，使用者不需要重新进行链接。
缺点
移植性不好。
速度会稍微慢一点
得知道根据优缺点，怎么去选动态库和静态库
一般根据移植性，体积来选
要知道怎么选，但是一般自己选的概率不大
生成静态库和动态库的指令
看一看，练一练
下次碰到了查文档
Makefile
需要知道什么是Makefile
就是一个脚本。我们用来生成可执行程序的命令，可以放在这个脚本里。这样我们就不用每次都去写这个生成的命令。
然后我们又想用这个命令。
脚本就是文本。 这个文本可以直接运行
Makefile的基本组成等元得知道 —— 基本语法得知道
增量编译得知道 —— 代码如果经过了更改， 它会重新执行编译这个步骤，最终生成main.o.
使用得知道
我给一个Makefile， 你要知道怎么使用
Makefile:5: *** missing separator. Stop.
一定要将前面的空格删掉。 再写tab。 一定哦一次
后续，我们的使用场景基本是怎样的？
后续一般是这样的，我们写一个test1.c文件
gcc test1.c -o test1 -Wall -g
所以，我们给大家提供了一个通用的Makefile。 检查你这里面有哪些.c文件。
然后将这些.c生成同名的 可执行程序。
只要这些文件做了修改， 就增量编译
通配符
test1.c --> test1
test2.c --> test2
Makefile的使用
创建一个Makefile，然后将我们写好的东西放进去。 它就可以帮你完成编译、链接的功能
同时也可以删除，或者重新编译链接这些文件
make
make clean
make rebuild
先把之前生成的文件，全干掉
然后再重新生成一遍
make buildagain
工作过程中，一般写Makefile的机会不多。